

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по домашнему заданию
“Игра 2048 на Unity”

Выполнил:
Студент группы ИУ5-34Б
Верзаков Н.В.
Преподаватель:
Нардид А.Н.

Москва 2025

Задание

1. Выберите язык программирования (который Вы ранее не изучали) и (1) напишите по нему реферат с примерами кода или (2) реализуйте на нем небольшой проект (с детальным текстовым описанием).
2. Реферат (проект) может быть посвящен отдельному аспекту (аспектам) языка или содержать решение какой-либо задачи на этом языке.
3. Необходимо установить на свой компьютер компилятор (интерпретатор, транспилятор) этого языка и произвольную среду разработки.
4. В случае написания реферата необходимо разработать и откомпилировать примеры кода (или модифицировать стандартные примеры).
5. В случае создания проекта необходимо детально комментировать код.
6. При написании реферата (создании проекта) необходимо изучить и корректно использовать особенности парадигмы языка и основных конструкций данного языка.
7. Приветствуется написание черновика статьи по результатам выполнения ДЗ. Черновик статьи может быть подготовлен группой студентов, которые исследовали один и тот же аспект в нескольких языках или решили одинаковую задачу на нескольких языках.

Описания, тексты программ

Скрипты, используемые в проекте Unity:

TileState.cs

```
using UnityEngine;

// ScriptableObject, определяющий визуальное состояние плитки (цвета для разных значений)
[CreateAssetMenu(menuName = "Tile State")]
public class TileState : ScriptableObject
{
    // Числовое значение, соответствующее этому состоянию (используется для идентификации)
    public int number;
    // Цвет фона плитки
    public Color backgroundColor;
    // Цвет текста на плитке
    public Color textColor;
}
```

Tile.cs

```
using System.Collections;
using TMPro;
using UnityEngine;
using UnityEngine.UI;

// Класс, представляющий отдельную плитку на игровой доске
public class Tile : MonoBehaviour
{
    public TileState State { get; private set; }
    public TileCell Cell { get; private set; }
    public int Number { get; private set; }
    public bool Locked { get; set; }
    private Image background;
    private TextMeshProUGUI text;

    // Инициализация компонентов при создании объекта
    private void Awake()
    {
        background = GetComponent<Image>();
        text = GetComponentInChildren<TextMeshProUGUI>();
    }

    // Устанавливает состояние плитки и обновляет визуальное отображение
    public void SetState(TileState state, int number)
    {
        this.State = state;
        this.Number = number;

        // Обновление визуального представления
    }
}
```

```

        background.color = state.backgroundColor;
        text.color = state.textColor;
        text.text = number.ToString();
    }

    // Начальное появление плитки в указанной ячейке без анимации
    public void Spawn(TileCell cell)
    {
        // Освобождаем предыдущую ячейку, если она была
        if (this.Cell != null)
        {
            this.Cell.Tile = null;
        }

        // Устанавливаем новую ячейку
        this.Cell = cell;
        this.Cell.Tile = this;

        // Мгновенное появление без анимации
        transform.position = cell.transform.position;
    }

    // Перемещает плитку в указанную ячейку с анимацией
    public void MoveTo(TileCell cell)
    {
        // Освобождаем предыдущую ячейку
        if (this.Cell != null)
        {
            this.Cell.Tile = null;
        }

        this.Cell = cell;
        this.Cell.Tile = this;

        // Запускаем анимацию перемещения
        StartCoroutine(Animate(cell.transform.position, false));
    }

    // Объединяет эту плитку с плиткой в указанной ячейке
    public void Merge(TileCell cell)
    {
        // Освобождаем текущую ячейку
        if (this.Cell != null)
        {
            this.Cell.Tile = null;
        }
        this.Cell = null;

        // Блокируем целевую плитку от повторного слияния
        cell.Tile.Locked = true;

        // Запускаем анимацию слияния (плитка будет уничтожена после анимации)
        StartCoroutine(Animate(cell.transform.position, true));
    }

```

```

    }

    // Корутина для анимации перемещения плитки
    private IEnumerator Animate(Vector3 to, bool merging)
    {
        float elapsed = 0f;
        float duration = 0.1f;

        Vector3 from = transform.position;

        while (elapsed < duration)
        {
            transform.position = Vector3.Lerp(from, to, elapsed / duration);
            elapsed += Time.deltaTime;
            yield return null;
        }

        transform.position = to;

        // Уничтожение плитки после слияния
        if (merging)
        {
            Destroy(gameObject);
        }
    }
}

```

TileCell.cs

```

using UnityEngine;

// Представляет одну ячейку на игровой доске
public class TileCell : MonoBehaviour
{
    public Vector2Int Coordinates { get; set; }
    public Tile Tile { get; set; }

    public bool Empty => Tile == null;
    public bool Occupied => Tile != null;
}

```

TileRow.cs

```

using UnityEngine;

// Представляет одну строку ячеек на игровой доске
public class TileRow : MonoBehaviour
{
    public TileCell[] Cells { get; private set; }

    // Инициализация ячеек строки при создании объекта
    private void Awake()
    {
        Cells = GetComponentsInChildren<TileCell>();
    }
}

```

```

    }

}

```

TileGrid.cs

```

using UnityEngine;

// Управляет сеткой ячеек игровой доски и предоставляет методы для работы с ячейками
public class TileGrid : MonoBehaviour
{
    public TileRow[] Rows { get; private set; }
    public TileCell[] Cells { get; private set; }

    public int Size => Cells.Length;
    public int Height => Rows.Length;
    public int Width => Size / Height;

    // Инициализация компонентов и установка координат ячеек
    private void Awake()
    {
        Rows = GetComponentInChildren<TileRow>();
        Cells = GetComponentInChildren<TileCell>();

        // Устанавливаем координаты ячеек на основе их индекса
        for (int i = 0; i < Cells.Length; i++)
        {
            Cells[i].Coordinates = new Vector2Int(i % Width, i / Width);
        }
    }

    // Дополнительная инициализация координат через структуру строк
    private void Start()
    {
        // Устанавливаем координаты через структуру строк для надежности
        for (int y = 0; y < Rows.Length; y++)
        {
            for (int x = 0; x < Rows[y].Cells.Length; x++)
            {
                Rows[y].Cells[x].Coordinates = new Vector2Int(x, y);
            }
        }
    }

    // Получает ячейку по координатам x и y
    public TileCell GetCell(int x, int y)
    {
        if (x >= 0 && x < Width && y >= 0 && y < Height)
        {
            return Rows[y].Cells[x];
        }
        else
        {

```

```

        return null;
    }
}

// Получает ячейку по координатам Vector2Int
public TileCell GetCell(Vector2Int coordinates)
{
    return GetCell(coordinates.x, coordinates.y);
}

// Получает соседнюю ячейку относительно указанной в заданном направлении
public TileCell GetAdjacentCell(TileCell cell, Vector2Int direction)
{
    Vector2Int coordinates = cell.Coordinates;
    coordinates.x += direction.x;
    coordinates.y -= direction.y; // Инвертируем Y, так как в Unity Y растет вверх

    return GetCell(coordinates);
}

// Находит случайную пустую ячейку на сетке
public TileCell GetRandomEmptyCell()
{
    // Начинаем со случайного индекса
    int index = Random.Range(0, Cells.Length);
    int startingIndex = index;

    // Ищем первую пустую ячейку, начиная со случайной позиции
    while (Cells[index].Occupied)
    {
        index++;

        // Переходим к началу массива, если достигли конца
        if (index >= Cells.Length)
        {
            index = 0;
        }

        // Если вернулись к начальной позиции, значит все ячейки заняты
        if (index == startingIndex)
        {
            return null;
        }
    }

    return Cells[index];
}
}

```

TileBoard.cs

```

using System.Collections.Generic;
using System.Collections;

```

```

using UnityEngine;
using UnityEngine.InputSystem;
using Unity.VisualScripting;
using System.Data;

// Управляет игровой доской: обработка ввода, перемещение плиток, слияние и проверка
окончания игры
public class TileBoard : MonoBehaviour
{
    public GameManager gameManager;
    public Tile TilePrefab;
    public TileState[] TileStates;
    private TileGrid Grid;
    private List<Tile> Tiles;
    private bool waiting; // Флаг ожидания завершения анимации перед следующим ходом

    // Инициализация компонентов при создании объекта
    private void Awake()
    {
        Grid = GetComponentInChildren<TileGrid>();
        Tiles = new List<Tile>(16); // Максимум 16 плиток на доске 4x4
    }

    // Очищает доску: удаляет все плитки и освобождает ячейки
    public void ClearBoard()
    {
        // Освобождаем все ячейки
        foreach (var cell in Grid.Cells)
        {
            cell.Tile = null;
        }

        // Уничтожаем все плитки
        foreach (var tile in Tiles)
        {
            Destroy(tile.gameObject);
        }
        Tiles.Clear();
    }

    // Создает новую плитку со значением 2 в случайной пустой ячейке
    public void CreateTile()
    {
        Tile tile = Instantiate(TilePrefab, Grid.transform);
        tile.SetState(TileStates[0], 2); // Первое состояние всегда соответствует
        значению 2
        tile.Spawn(Grid.GetRandomEmptyCell());
        Tiles.Add(tile);
    }

    // Обработка ввода с клавиатуры для перемещения плиток
    private void Update()
    {

```

```

        if (!waiting)
        {
            // Движение вверх (W или стрелка вверх)
            if (Keyboard.current.wKey.wasPressedThisFrame ||
Keyboard.current.upArrowKey.wasPressedThisFrame)
            {
                MoveTiles(Vector2Int.up, 0, 1, 1, 1);
            }
            // Движение влево (A или стрелка влево)
            else if (Keyboard.current.aKey.wasPressedThisFrame ||
Keyboard.current.leftArrowKey.wasPressedThisFrame)
            {
                MoveTiles(Vector2Int.left, 1, 1, 0, 1);
            }
            // Движение вниз (S или стрелка вниз)
            else if (Keyboard.current.sKey.wasPressedThisFrame ||
Keyboard.current.downArrowKey.wasPressedThisFrame)
            {
                MoveTiles(Vector2Int.down, 0, 1, Grid.Height - 2, -1);
            }
            // Движение вправо (D или стрелка вправо)
            else if (Keyboard.current.dKey.wasPressedThisFrame ||
Keyboard.current.rightArrowKey.wasPressedThisFrame)
            {
                MoveTiles(Vector2Int.right, Grid.Width - 2, -1, 0, 1);
            }
        }
    }

    // Перемещает все плитки в указанном направлении
    private void MoveTiles(Vector2Int direction, int startX, int incrementX, int
startY, int incrementY)
    {
        bool changed = false;

        // Обход ячеек в правильном порядке для корректного перемещения
        for (int x = startX; x >= 0 && x < Grid.Width; x += incrementX)
        {
            for (int y = startY; y >= 0 && y < Grid.Height; y += incrementY)
            {
                TileCell cell = Grid.GetCell(x, y);

                // Перемещаем плитку, если ячейка занята
                if (cell.Occupied)
                {
                    changed |= MoveTile(cell.Tile, direction);
                }
            }
        }

        // Если были изменения, ждем завершения анимации и создаем новую плитку
        if (changed)
        {

```

```

        StartCoroutine(WaitForChanges());
    }
}

// Перемещает одну плитку в указанном направлении
private bool MoveTile(Tile tile, Vector2Int direction)
{
    TileCell newCell = null;
    TileCell adjacent = Grid.GetAdjacentCell(tile.Cell, direction);

    // Ищем самую дальнюю доступную ячейку в направлении движения
    while (adjacent != null)
    {
        if (adjacent.Occupied)
        {
            // Если ячейка занята, проверяем возможность слияния
            if (CanMerge(tile, adjacent.Tile))
            {
                Merge(tile, adjacent.Tile);
                return true;
            }

            // Если слияние невозможно, останавливаемся
            break;
        }

        newCell = adjacent;
        adjacent = Grid.GetAdjacentCell(adjacent, direction);
    }

    // Перемещаем плитку в найденную ячейку
    if (newCell != null)
    {
        tile.MoveTo(newCell);
        return true;
    }

    return false;
}

// Проверяет, могут ли две плитки быть объединены
private bool CanMerge(Tile a, Tile b)
{
    return a.Number == b.Number && !b.Locked;
}

// Объединяет две плитки: уничтожает первую и удваивает значение второй
private void Merge(Tile a, Tile b)
{
    Tiles.Remove(a);
    a.Merge(b.Cell);

    // Находим следующее состояние для объединенной плитки

```

```

    int index = Mathf.Clamp(IndexOf(b.State) + 1, 0, TileStates.Length - 1);
    int number = b.Number * 2;

    // Обновляем состояние и значение плитки
    b.SetState(TileStates[index], number);

    // Увеличиваем счет на значение объединенной плитки
    gameManager.IncreaseScore(number);
}

// Находит индекс состояния плитки в массиве TileStates
private int IndexOf(TileState state)
{
    for (int i = 0; i < TileStates.Length; i++)
    {
        if (state == TileStates[i])
        {
            return i;
        }
    }

    return -1;
}

// Корутина ожидания завершения анимации перед следующим действием
private IEnumerator WaitForChanges()
{
    waiting = true;

    // Ждем завершения анимации
    yield return new WaitForSeconds(0.1f);

    waiting = false;

    // Разблокируем все плитки для следующего хода
    foreach (var tile in Tiles)
    {
        tile.Locked = false;
    }

    // Создаем новую плитку, если есть свободные ячейки
    if (Tiles.Count != Grid.Size)
    {
        CreateTile();
    }

    // Проверяем условие окончания игры
    if (CheckForGameOver())
    {
        gameManager.GameOver();
    }
}

```

```

// Проверяет, закончилась ли игра (нет возможных ходов)
private bool CheckForGameOver()
{
    // Если есть свободные ячейки, игра не окончена
    if (Tiles.Count != Grid.Size)
    {
        return false;
    }

    // Проверяем каждую плитку на возможность слияния с соседями
    foreach (var tile in Tiles)
    {
        TileCell up = Grid.GetAdjacentCell(tile.Cell, Vector2Int.up);
        TileCell left = Grid.GetAdjacentCell(tile.Cell, Vector2Int.left);
        TileCell down = Grid.GetAdjacentCell(tile.Cell, Vector2Int.down);
        TileCell right = Grid.GetAdjacentCell(tile.Cell, Vector2Int.right);

        // Если хотя бы одно слияние возможно, игра продолжается
        if (up != null && CanMerge(tile, up.Tile))
        {
            return false;
        }
        if (left != null && CanMerge(tile, left.Tile))
        {
            return false;
        }
        if (down != null && CanMerge(tile, down.Tile))
        {
            return false;
        }
        if (right != null && CanMerge(tile, right.Tile))
        {
            return false;
        }
    }

    // Нет возможных ходов – игра окончена
    return true;
}
}

```

GameManager.cs

```

using System.Collections;
using UnityEngine;
using TMPro;

// Главный менеджер игры, управляющий состоянием игры, счетом и UI
public class GameManager : MonoBehaviour
{
    public TileBoard board;
    public CanvasGroup gameOver;
    public TextMeshProUGUI scoreText;
}

```

```

public TextMeshProUGUI bestScore;
private int score;

// Инициализация новой игры при старте
private void Start()
{
    NewGame();
}

// Начинает новую игру: сбрасывает счет, очищает доску и создает начальные плитки
public void NewGame()
{
    SetScore(0);
    bestScore.text = LoadBestScore().ToString();

    gameOver.alpha = 0f;
    gameOver.interactable = false;

    board.ClearBoard();
    board.CreateTile();
    board.CreateTile();
    board.enabled = true;
}

// Обрабатывает окончание игры: отключает доску и показывает экран окончания
public void GameOver()
{
    board.enabled = false;
    gameOver.interactable = true;

    StartCoroutine(Fade(gameOver, 1f, 1f));
}

// Плавное изменение прозрачности с задержкой
private IEnumerator Fade(CanvasGroup canvasGroup, float to, float delay)
{
    yield return new WaitForSeconds(delay);

    float elapsed = 0f;
    float duration = 0.5f;
    float from = canvasGroup.alpha;

    // Плавная интерполяция прозрачности
    while (elapsed < duration)
    {
        canvasGroup.alpha = Mathf.Lerp(from, to, elapsed / duration);
        elapsed += Time.deltaTime;
        yield return null;
    }

    canvasGroup.alpha = to;
}

public void IncreaseScore(int points)
{

```

```

        SetScore(score + points);
    }
    private void SetScore(int score)
    {
        this.score = score;
        scoreText.text = score.ToString();
        SaveBestScore();
    }
    private void SaveBestScore()
    {
        int bestscore = LoadBestScore();
        if (score > bestscore)
        {
            PlayerPrefs.SetInt("bestscore", score);
        }
    }
    private int LoadBestScore()
    {
        return PlayerPrefs.GetInt("bestscore", 0);
    }
}

```

Работа игры



